

1 Inicialización del proyecto

1.1 Crear proyecto Node

bash

 Copiar

```
mkdir api-project  
cd api-project  
npm init -y
```

1.2 Instalar dependencias

bash

 Copiar

```
npm install express pg dotenv  
npm install --save-dev nodemon
```

Opcional (si quieres validaciones más robustas):

bash

 Copiar

```
npm install joi
```

2 Estructura de carpetas recomendada

código

 Copiar

```
/api-project
├── /src
│   ├── /config
│   │   └── db.js
│   ├── /routes
│   │   └── entity.routes.js
│   ├── /controllers
│   │   └── entity.controller.js
│   ├── /services (opcional pero recomendado)
│   │   └── entity.service.js
│   ├── /middlewares
│   │   └── error.middleware.js
│   └── app.js
├── .env
├── server.js
└── package.json
```

3 Configuración de conexión a PostgreSQL

3.1 Archivo .env

código

 Copiar

```
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=tu_password
DB_NAME=tu_database
```

3.2 Configuración de pool (db.js)

- Usar Pool de pg
- Exportar una instancia reutilizable
- Manejar errores de conexión

Esto permite centralizar la capa de acceso a datos.

4 Configuración del servidor

4.1 app.js

- Inicializar express
- Usar `express.json()`
- Registrar rutas
- Middleware global de errores

4.2 server.js

- Cargar `dotenv`
 - Levantar servidor con `app.listen`
 - Manejar errores de arranque
-

5 Implementación del CRUD (Punto 8)

Aquí es donde entra la mayor parte del tiempo (≈ 2.5 horas).

El flujo ideal es este:

5.1 Flujo lógico por entidad

Para cada entidad principal:

1. Crear archivo de rutas
 2. Crear controlador
 3. (Opcional) Crear service para separar lógica de negocio
 4. Implementar endpoints
-

5.2 Endpoints mínimos (CRUD completo)

Para una entidad resource:

Método	Endpoint	Acción
GET	/resource	Listar todos
GET	/resource/:id	Obtener por id
POST	/resource	Crear
PUT	/resource/:id	Actualizar
DELETE	/resource/:id	Eliminar

5.3 Orden recomendado de implementación

1 GET all

- Ejecutar query ya validada en DBeaver.
- Retornar 200 + array de resultados.
- Manejar error 500.

2 GET by id

- Validar que id sea número.
- Ejecutar SELECT con WHERE.
- Si no existe → 404.
- Si existe → 200.

3 POST

- Validar body.
- Validar campos obligatorios.
- Insertar con RETURNING *.
- Retornar 201.
- Manejar errores de constraint (FK, UNIQUE).

4 PUT

- Validar id.
- Verificar existencia previa.
- Actualizar.
- Retornar 200 o 204.

5 DELETE

- Validar id.
- Verificar existencia.
- Eliminar.
- Retornar 204.

6 Validaciones

Nivel intermedio esperado en prueba técnica:

- Validar tipos de datos
- Campos obligatorios
- Longitud de strings
- Validación de formato (email, fechas)
- Manejo de errores de base de datos

Si no usas librería externa, mínimo:

js

 Copiar

```
if (!name || typeof name !== "string") {  
  return res.status(400).json({ error: "Name" });  
}
```

7 Manejo centralizado de errores

Implementar middleware:

- Captura errores
- Retorna JSON consistente
- No exponer detalles internos de la base de datos

Ejemplo de estructura de respuesta:

json

 Copiar

```
{  
  "error": "Internal Server Error"  
}
```

8 Buenas prácticas que suman puntos

- ✓ Usar async/await
 - ✓ Try/catch en cada controlador
 - ✓ No repetir lógica de conexión
 - ✓ Separar SQL en variables claras
 - ✓ Usar códigos HTTP correctos
 - ✓ Estructura limpia y coherente
 - ✓ Nombres consistentes
-

9 Flujo mental recomendado durante la implementación

1. Implementar un endpoint completo.
2. Probarlo inmediatamente en Postman.
3. Ajustar.
4. Pasar al siguiente.

No implementes todo sin probar.

10 Secuencia realista de ejecución (2.5 horas)

1. Configurar conexión (15 min)
 2. Crear estructura base (10 min)
 3. Implementar primer CRUD completo (45–60 min)
 4. Implementar siguientes entidades más rápido (30–40 min cada una)
 5. Agregar validaciones finales (20 min)
 6. Testing general (20 min)
-

Resultado esperado al finalizar

- API funcional
- CRUD completo
- Validaciones mínimas
- Manejo correcto de errores
- Código organizado
- Lista para documentar en README

